

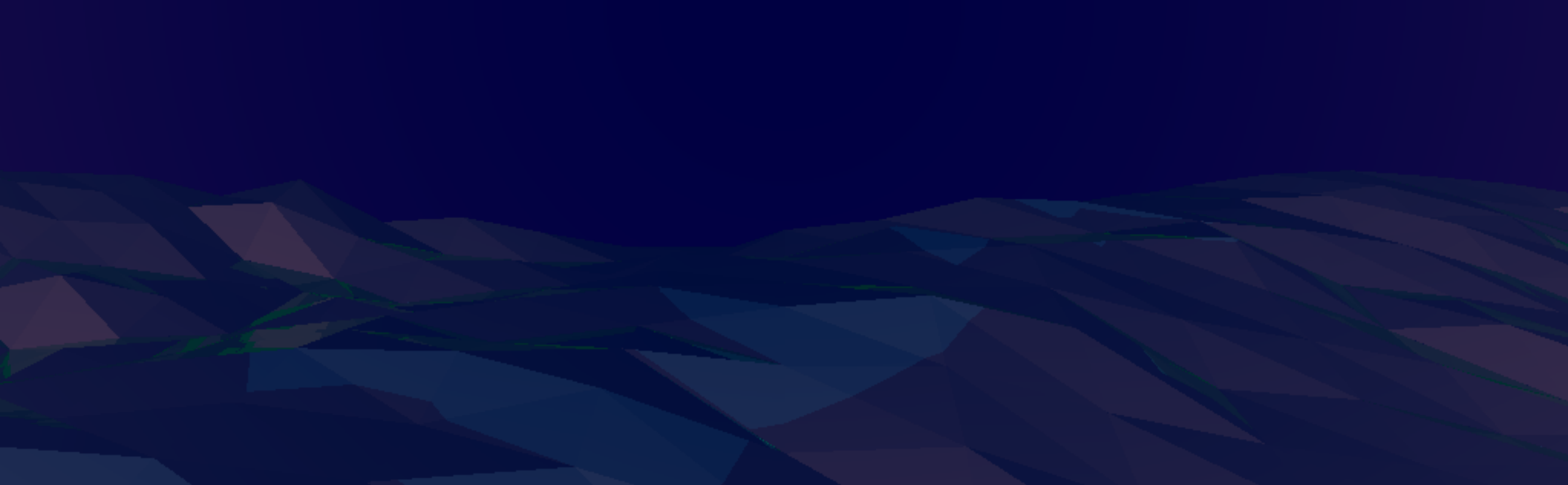


Sunset over an infinite sea

IGR Project

Alice Jeannin



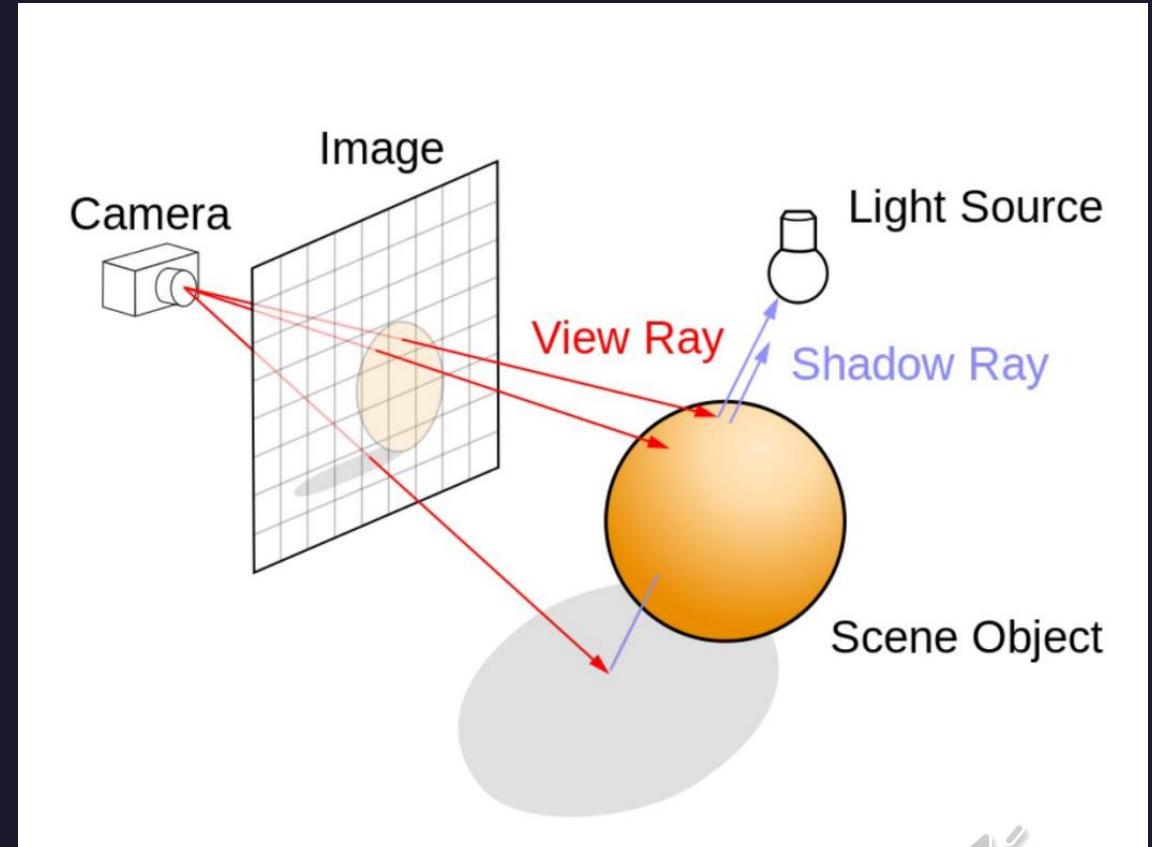


PART I – Ray tracing



Principle of ray tracing

```
shoot a ray
if (triangle intersected ){
    shoot reflected ray
    shoot transmitted ray
    shoot shadow ray
    return mixed color
}
else{
    return background color
}
```



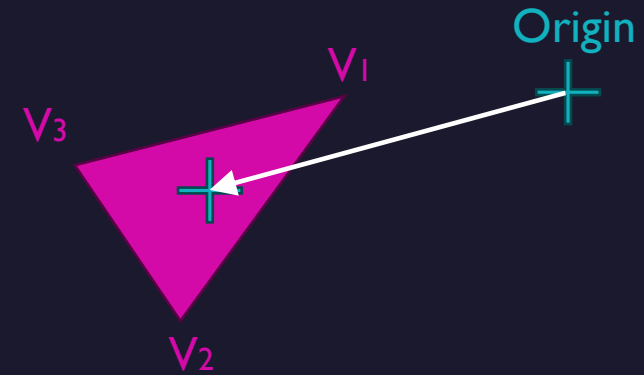
ray-triangle intersection : Möller-Trumbore algorithm

- Based on barycentric coordinates
- Uses Cramer's rule

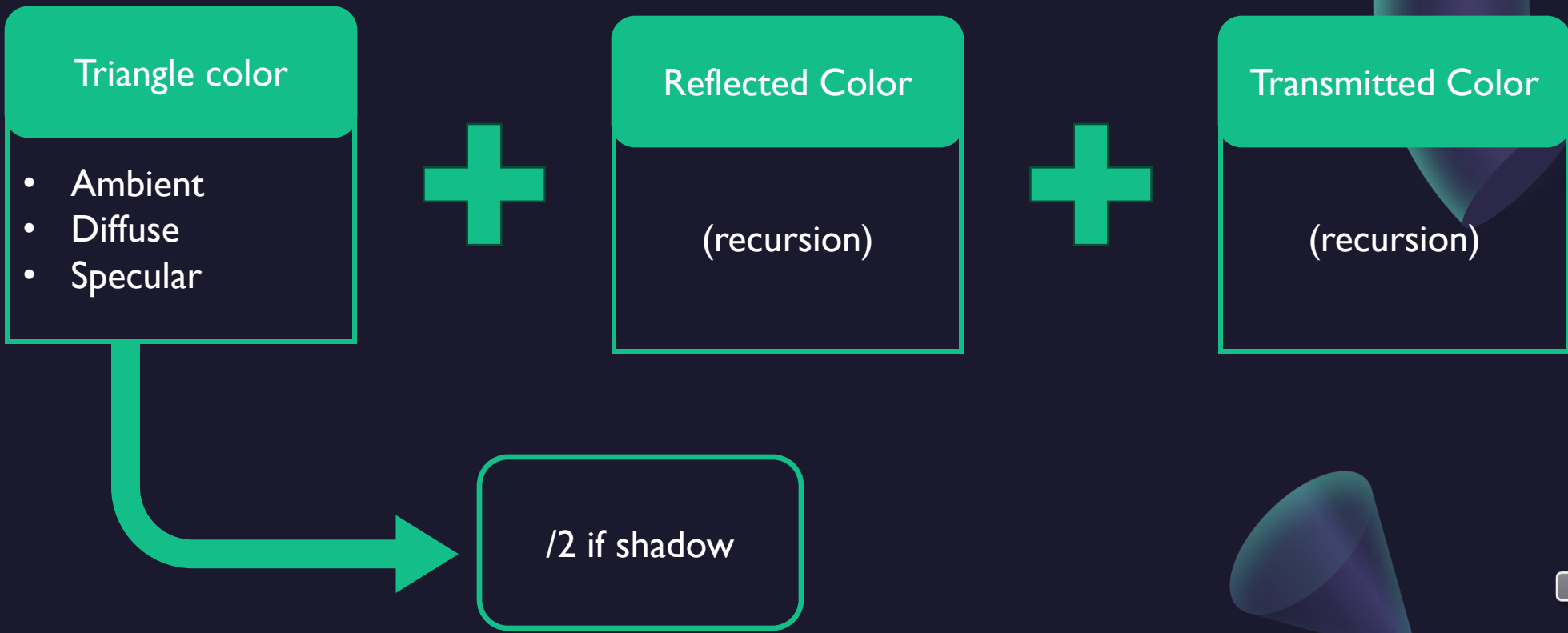
$$IntersectionPoint = (1 - u - v)V1 + u * V2 + v * V3 \quad (1)$$

$$IntersectionPoint = Origin + d * Ray \quad (2)$$

$$\begin{bmatrix} -Ray & (V2 - V1)(V3 - V1) \end{bmatrix} \begin{bmatrix} d \\ u \\ v \end{bmatrix} = Origin - V1 \quad (3)$$

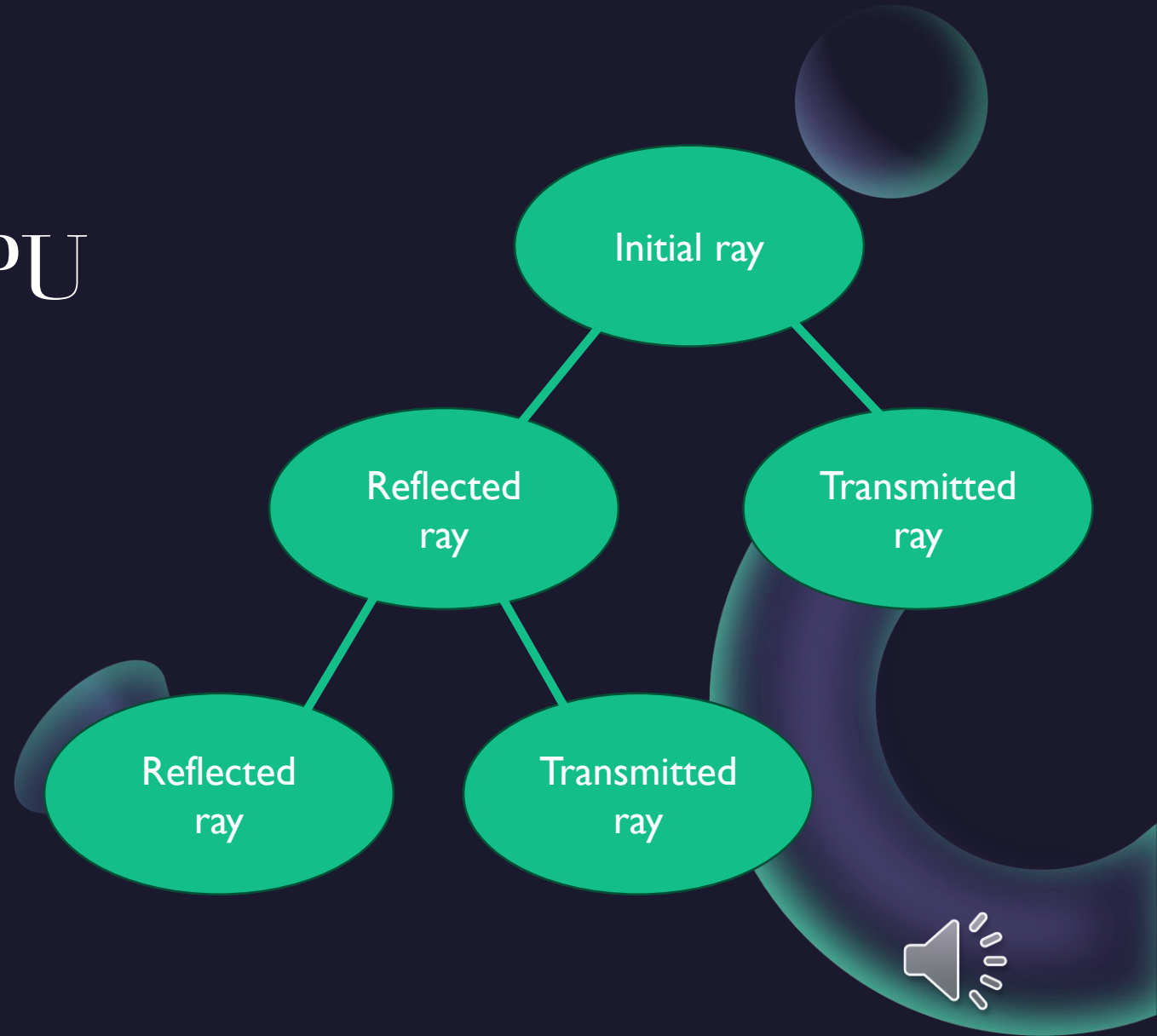


Computing color

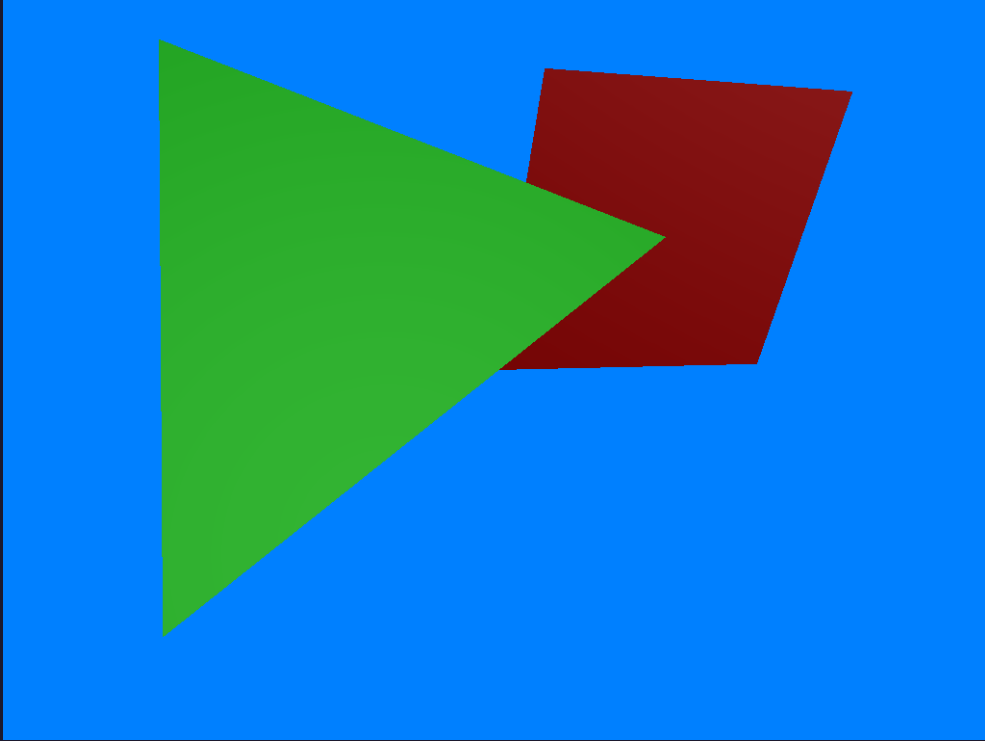


Why and how to use GPU

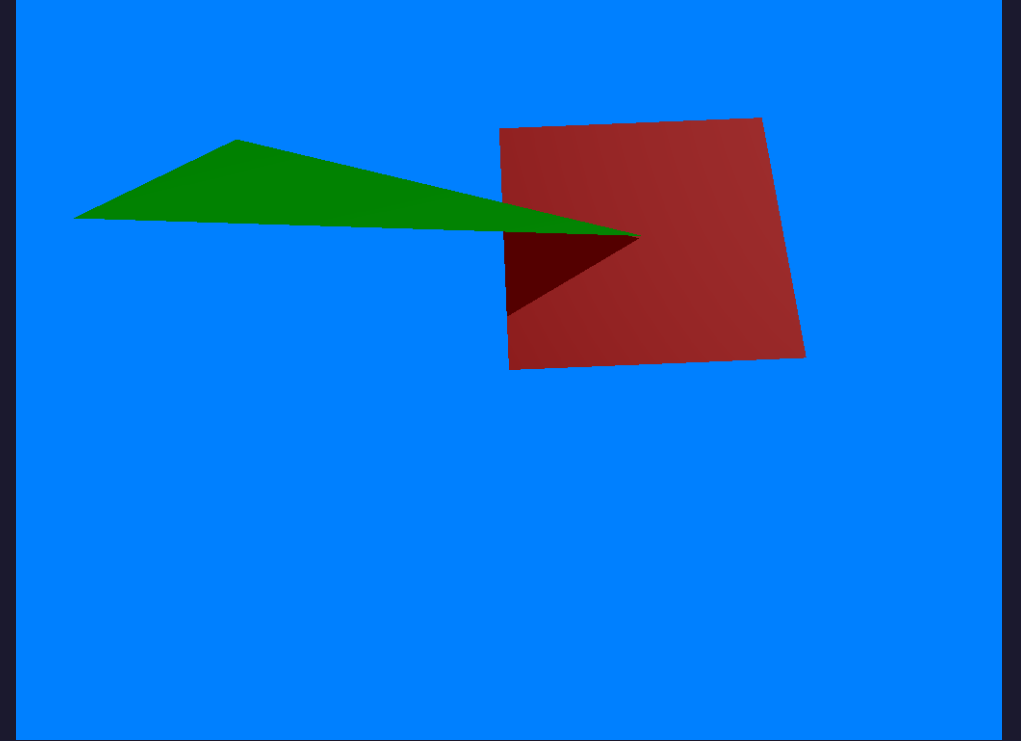
- Parallelism : initial ray for each pixels
- Vertex shader : 2 triangles for the screen
- Fragment shader : compute ray tracing
- Sending vertices to GPU : storage buffers
- No recursive function in glsl -> binary Tree



Results



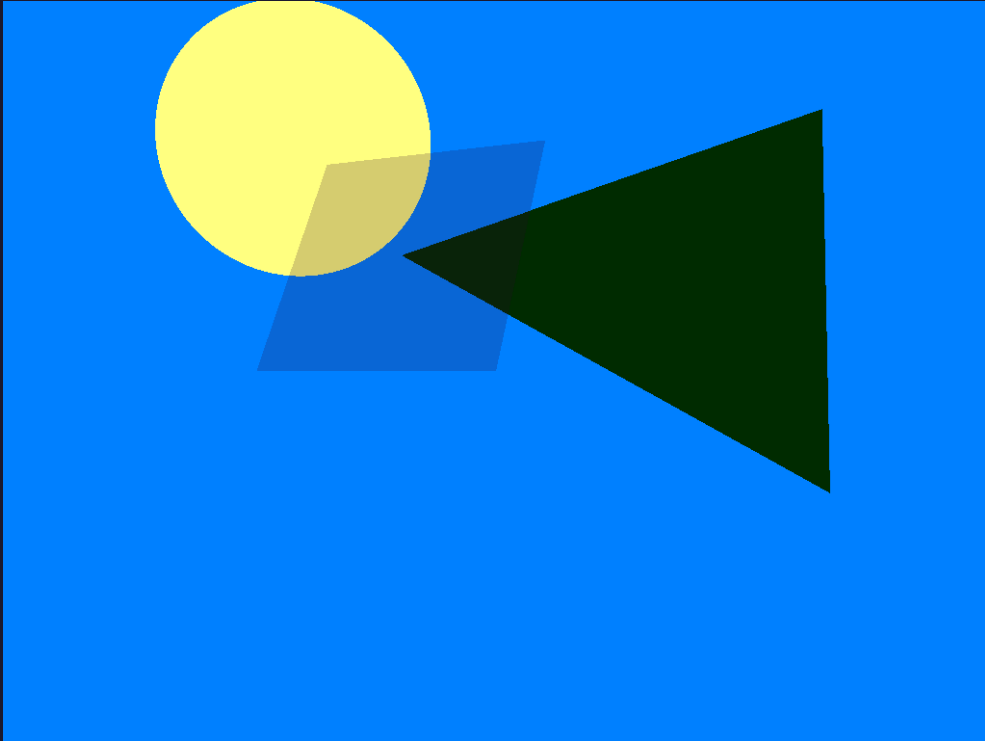
Lighting



Shadow



Results

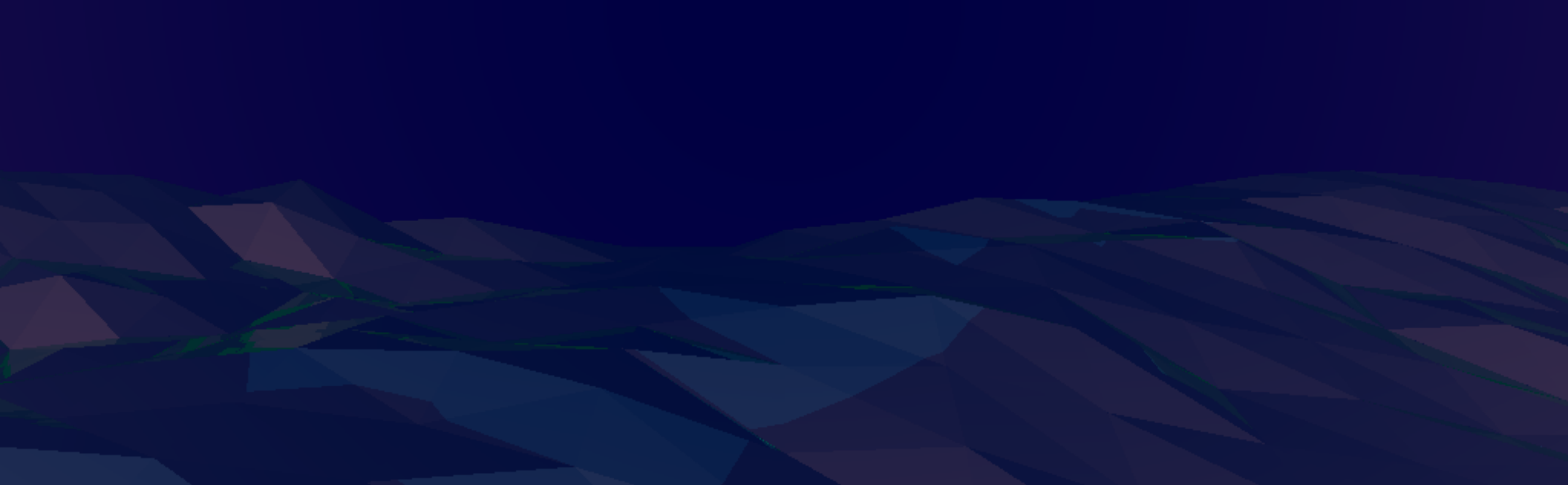


Transparency



Mirror

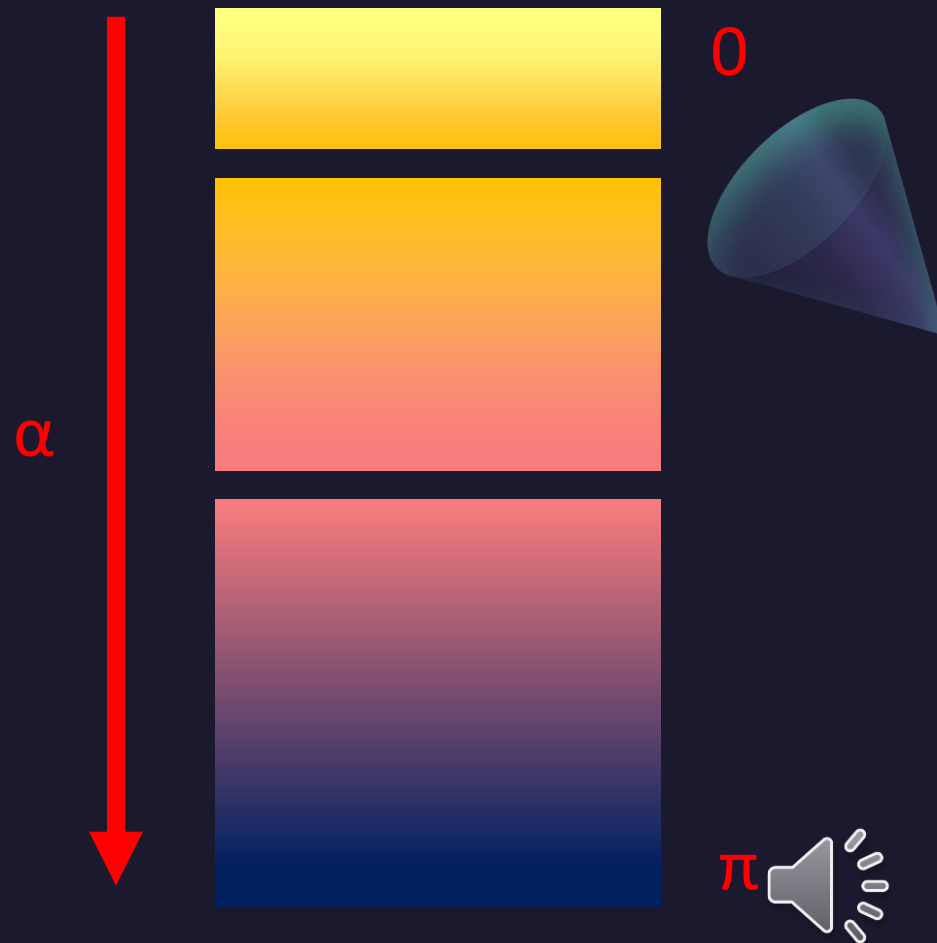
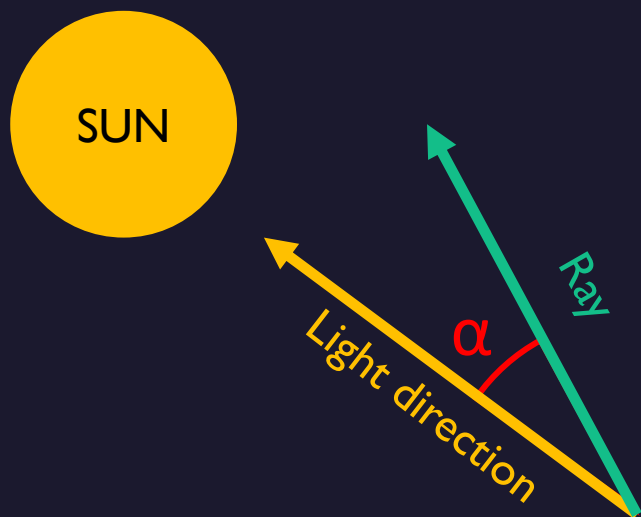




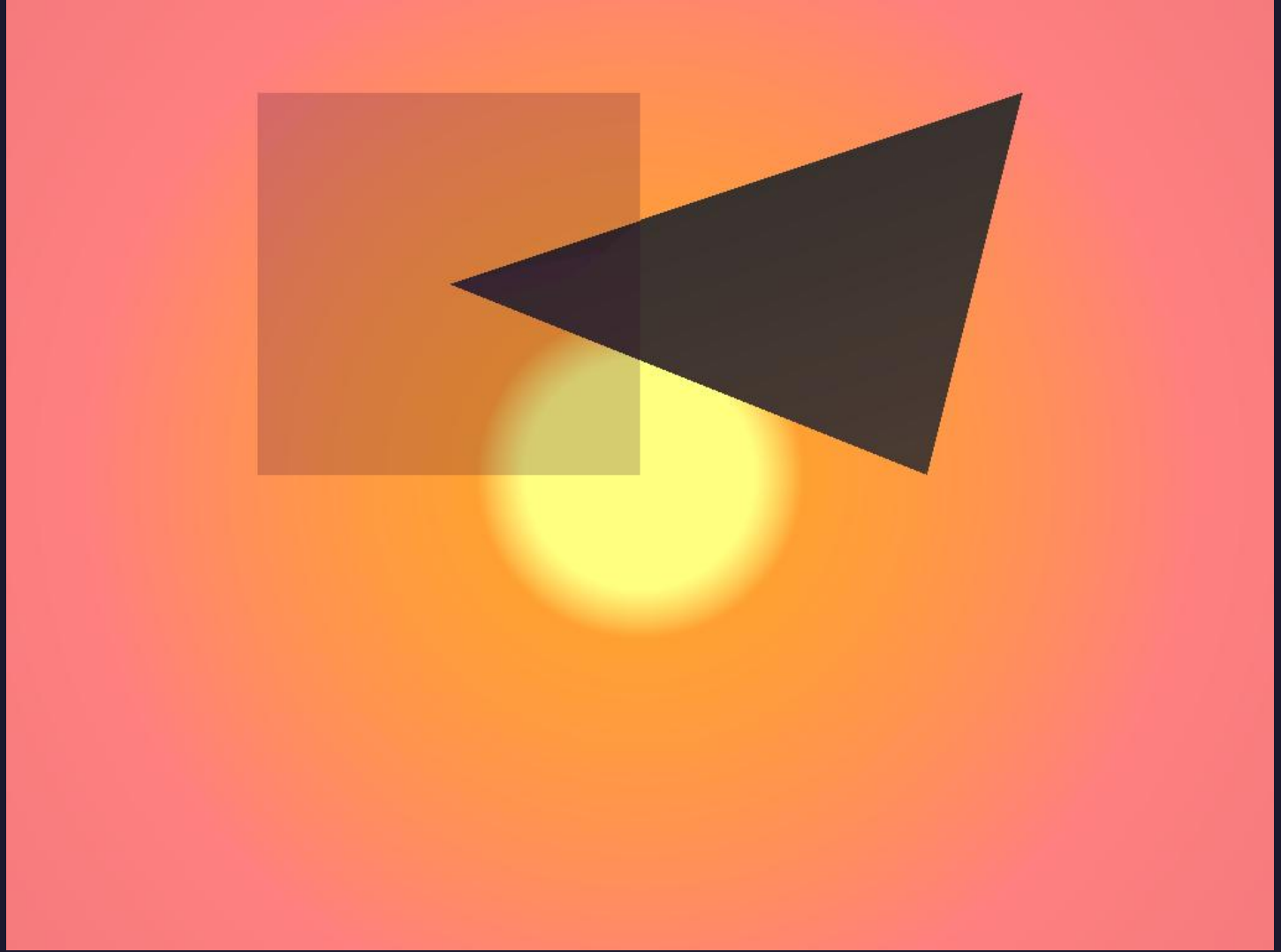
PART II – Background

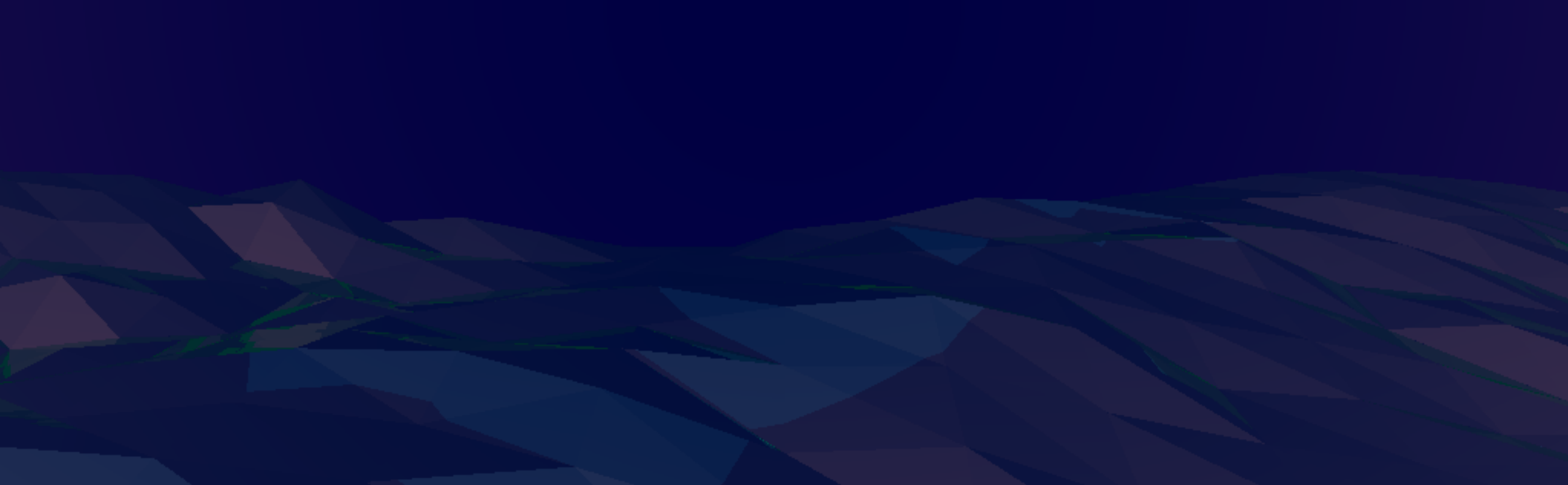


Sky with color gradient



Result





PART III – Sea geometry and animation



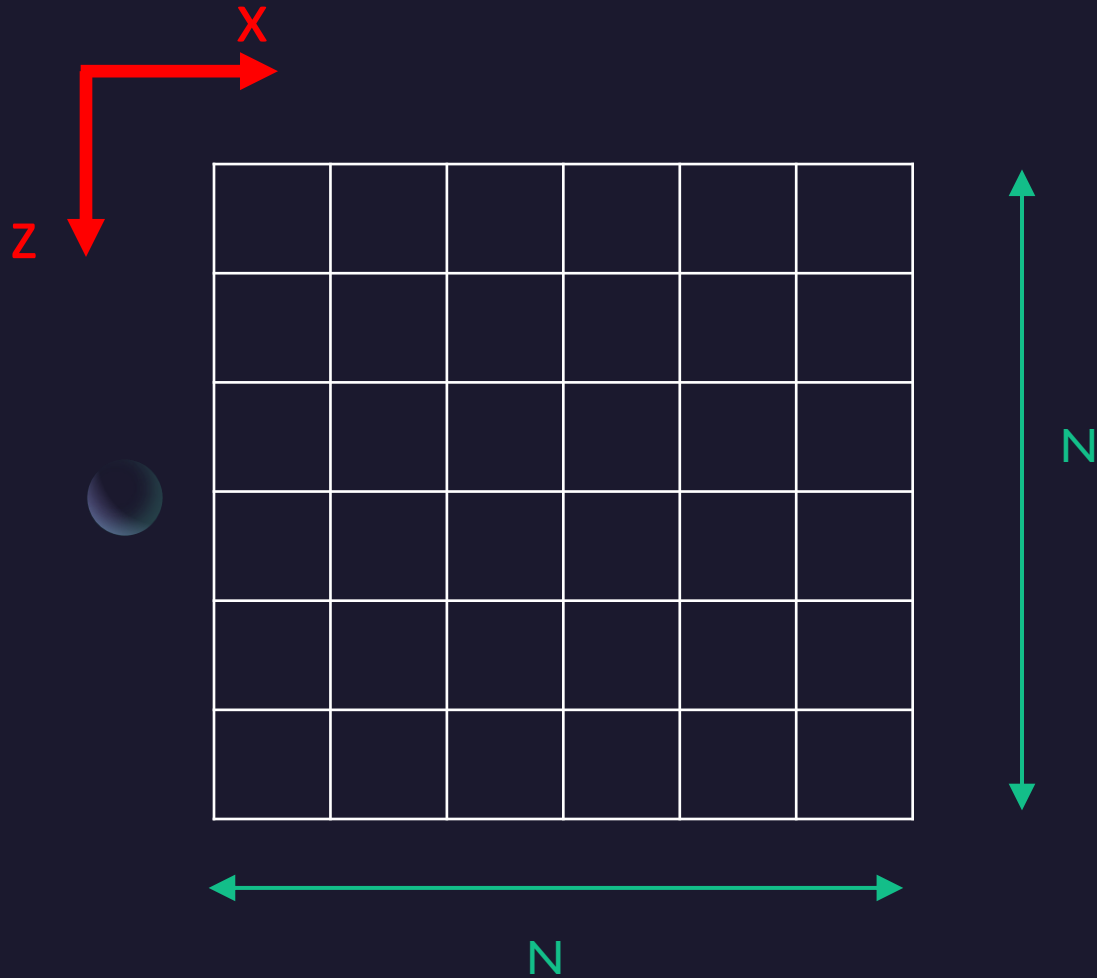


Source of inspiration : Monument valley

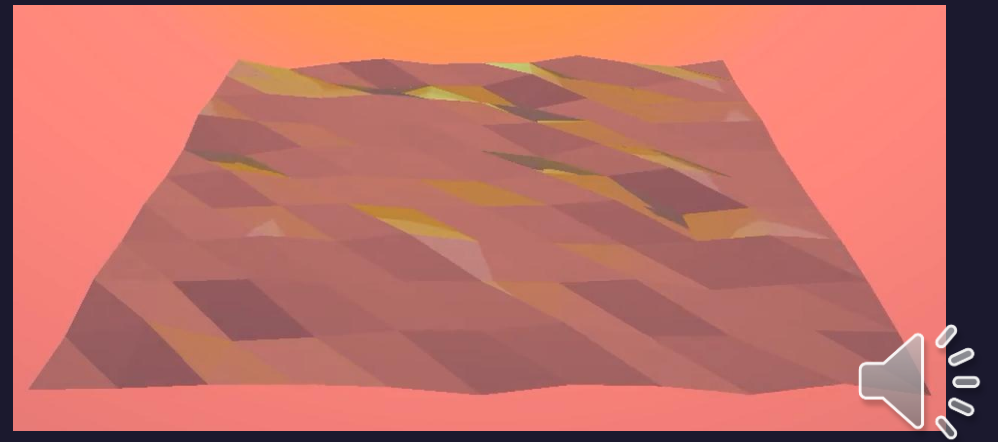
(or maybe a kind of reverse engineering...)



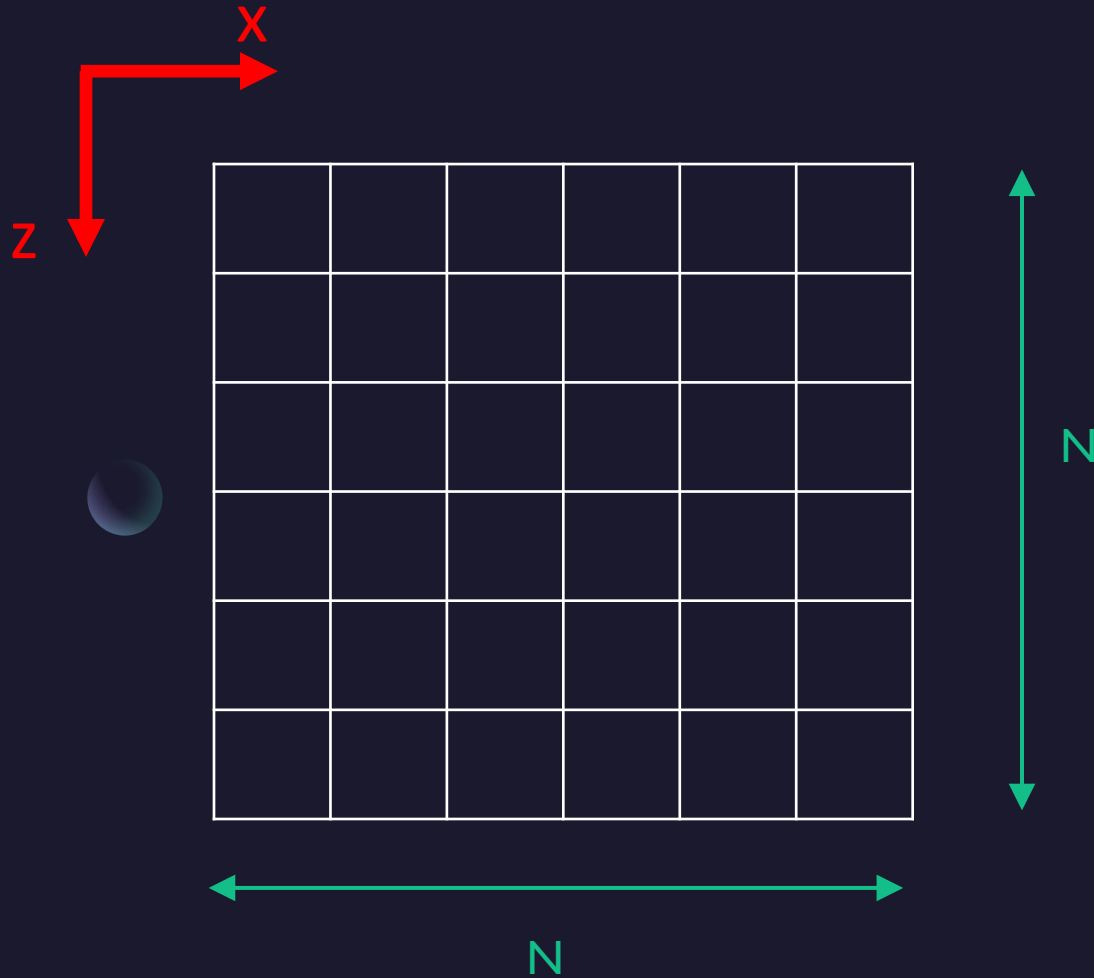
Sea geometry



- $(N+1)^2$ vertices
- $2 \cdot N^2$ triangles
- y coordinate : random value

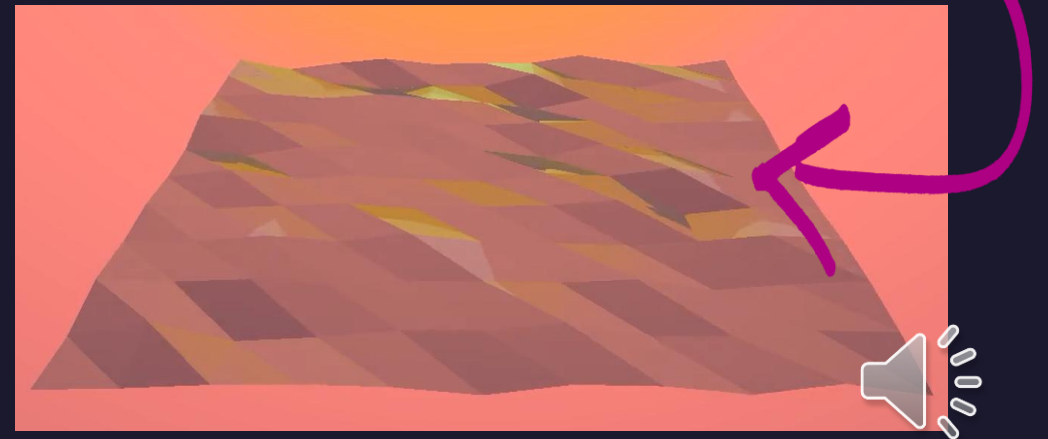


Sea geometry



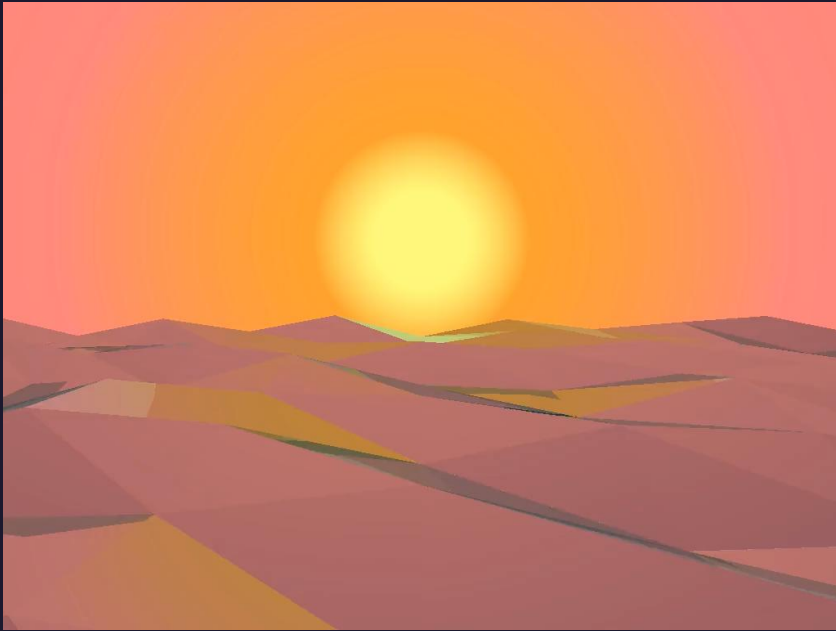
- $(N+1)^2$ vertices
- $2*N^2$ triangles
- y coordinate : random value

Color : cyan
70% of reflection



First animation : spring

- Each vertex attached to a vertical spring



Same spring constant for every vertex



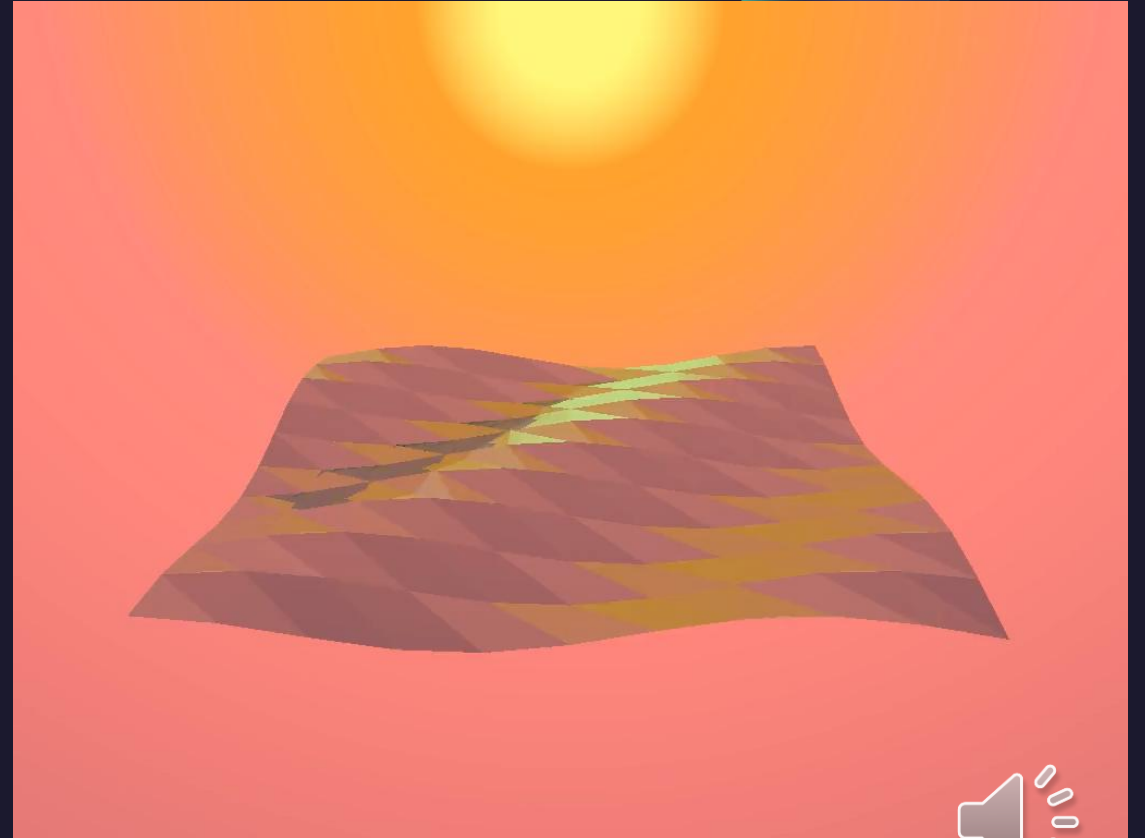
Random spring constant



Second animation : sinusoidal plane wave

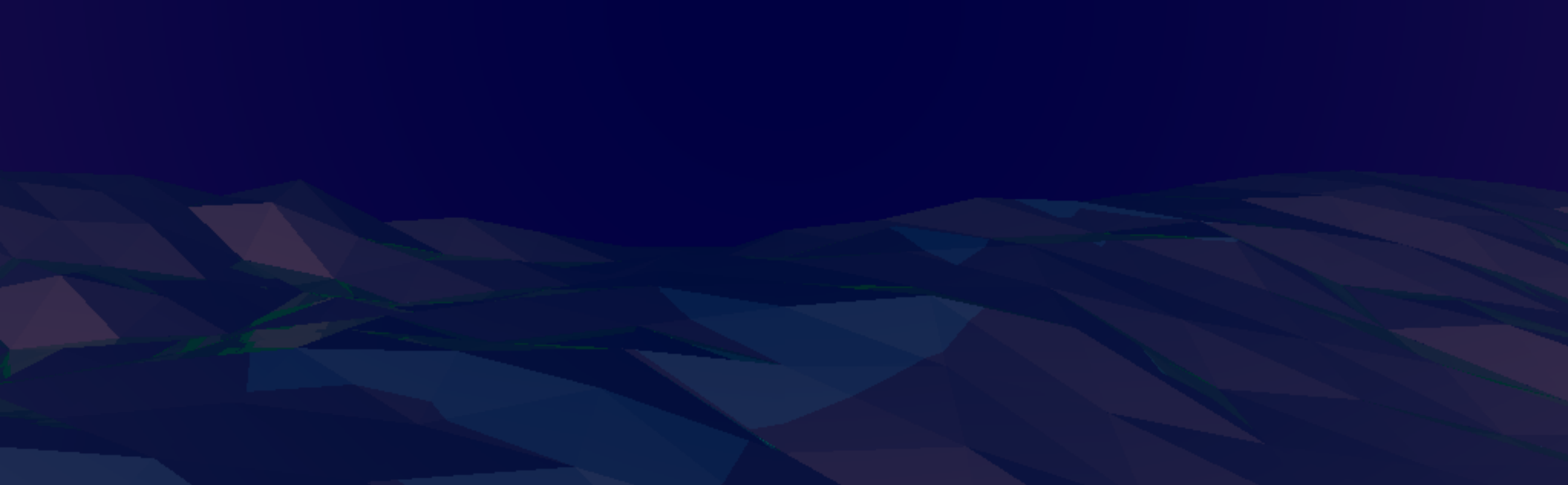
- Problem of random springs : no coherence between vertices
- Simulate wave : cosine!

$$y(t, x, z) = \cos(\omega t - \vec{k} \cdot \begin{bmatrix} x \\ z \end{bmatrix})$$



Compromise : sum of 2 heights





PART IV – Infinite sea





Idea : moving the
square with the
camera



Data structure

Vertex index

<i>0</i>	<i>1</i>	<i>2</i>
<i>3</i>	<i>4</i>	<i>5</i>
<i>6</i>	<i>7</i>	<i>8</i>

List of triangles :

(0, 1, 3), (1, 3, 4),

(1, 2, 4), (2, 4, 5),

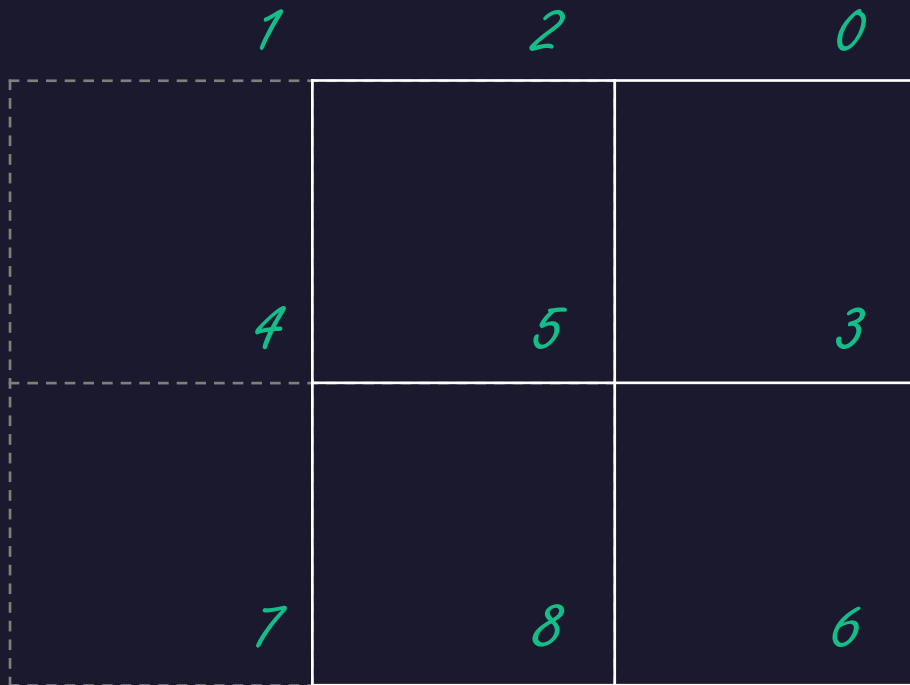
...

(4, 5, 7), (5, 7, 8)



Data structure

Vertex index



List of triangles :

~~(0, 1, 3)~~, ~~(1, 3, 4)~~,

(1, 2, 4), (2, 4, 5),

...

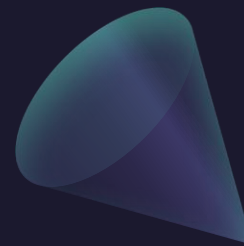
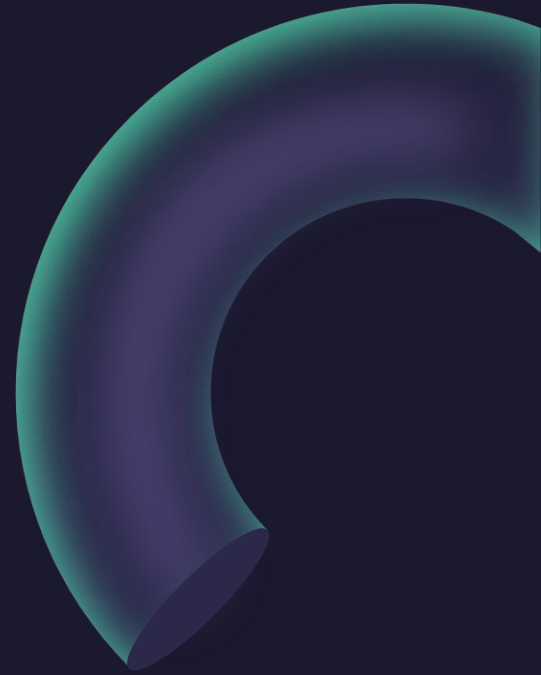
(4, 5, 7), (5, 7, 8),

(2, 0, 5), (0, 5, 3),

(5, 3, 8), (3, 8, 6)



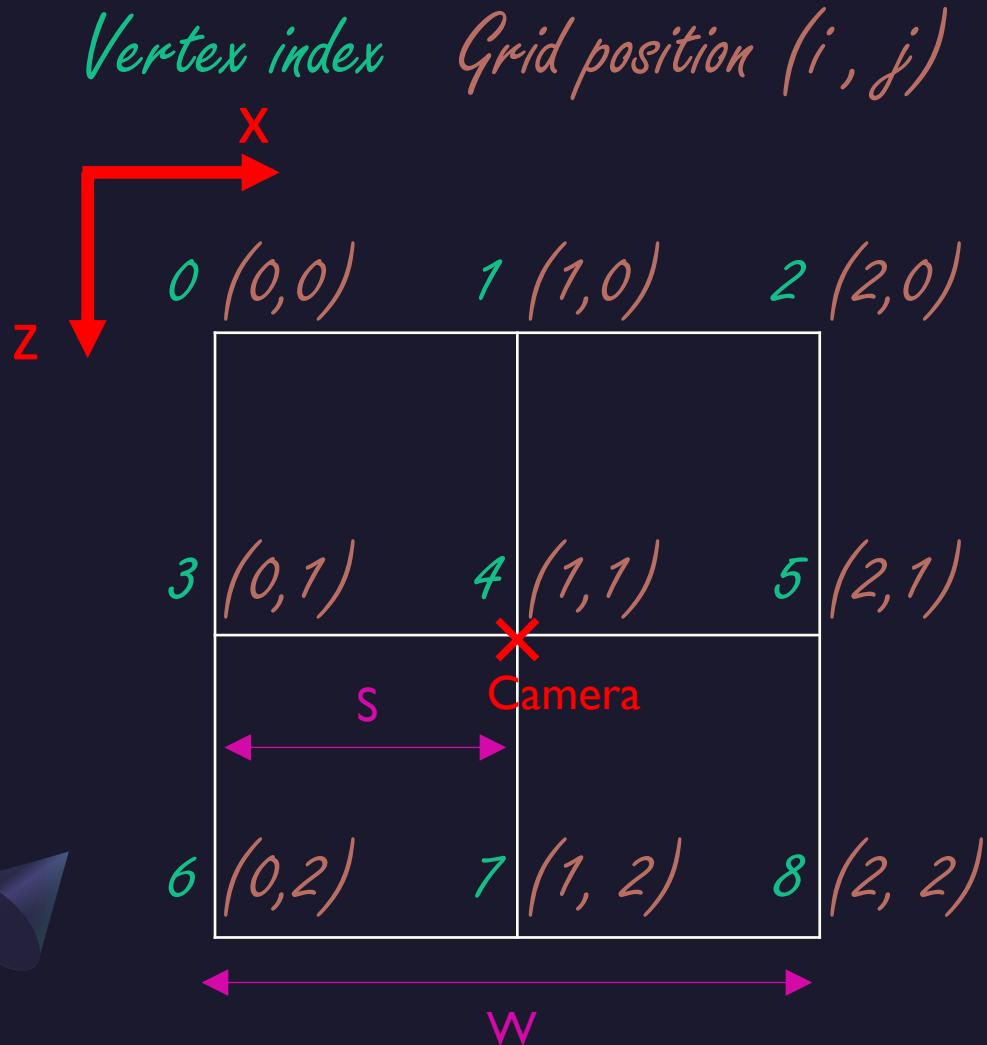
- How to know when to move vertices ?
- How to know which triangle to display ?
- How to update the list of triangles in an efficient way ?



Conventions (modulo)

$$a \% b = c \Leftrightarrow \begin{cases} a = k.b + c \\ k \in \mathbb{Z} \\ c \in [0, b[\end{cases}$$

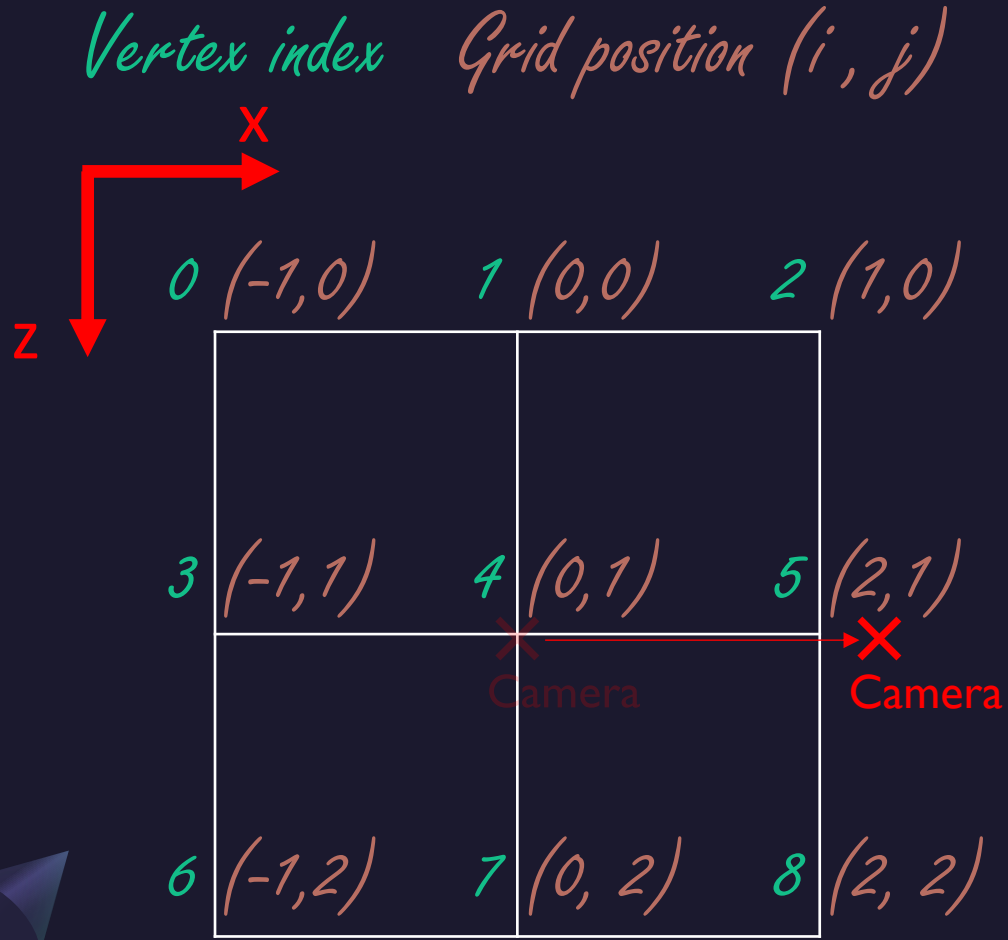
Computing grid positions (relative to camera position)



$$i = \text{round}((x - \text{camera}.x + \frac{W}{2})/S)$$
$$j = \text{round}((z - \text{camera}.z + \frac{W}{2})/S)$$



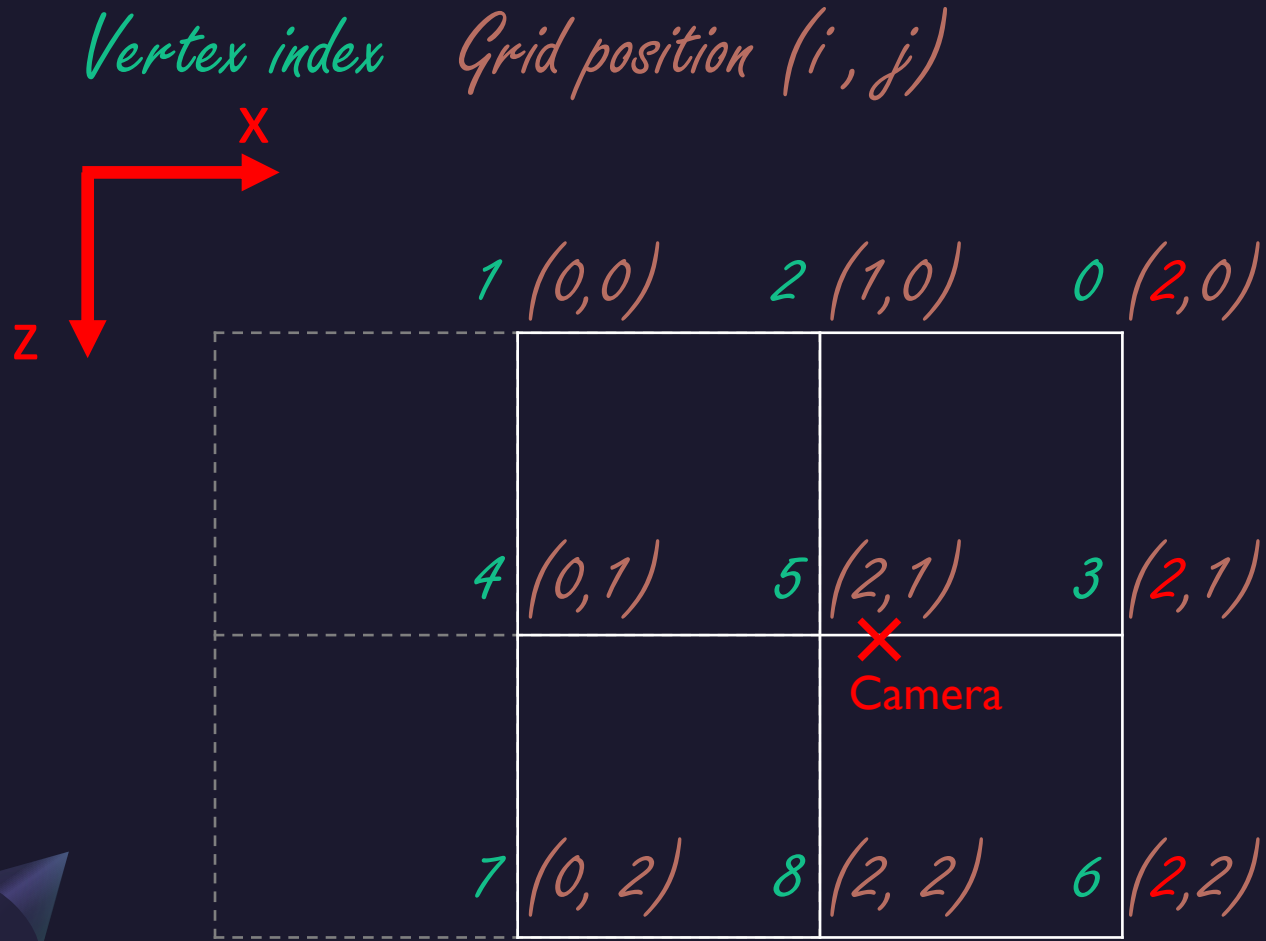
Computing grid positions (relative to camera position)



$$i = \text{round}((x - \text{camera}.x + \frac{W}{2})/S)$$
$$j = \text{round}((z - \text{camera}.z + \frac{W}{2})/S)$$



Computing grid positions (relative to camera position)



$$i = \text{round}((x - \text{camera}.x + \frac{W}{2})/S)$$
$$j = \text{round}((z - \text{camera}.z + \frac{W}{2})/S)$$

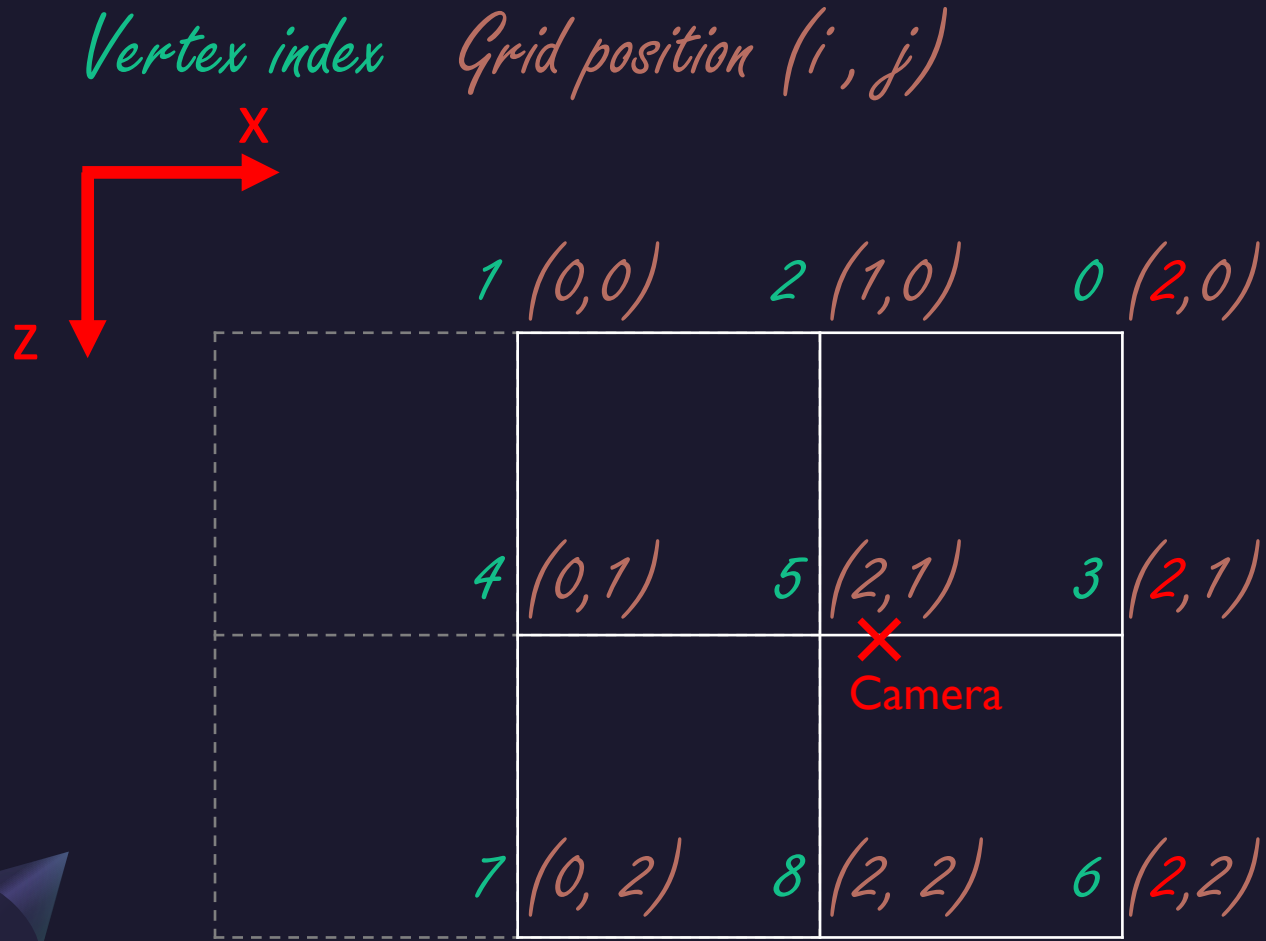
$$i' = i \% (N + 1)$$

$$j' = j \% (N + 1)$$

$$x' = x + \frac{i' - i}{N + 1} \cdot S \cdot (N + 1)$$

$$z' = z + \frac{j' - j}{N + 1} \cdot S \cdot (N + 1)$$

Computing grid positions (relative to camera position)



Move if $i' \neq i$ or $j' \neq j$

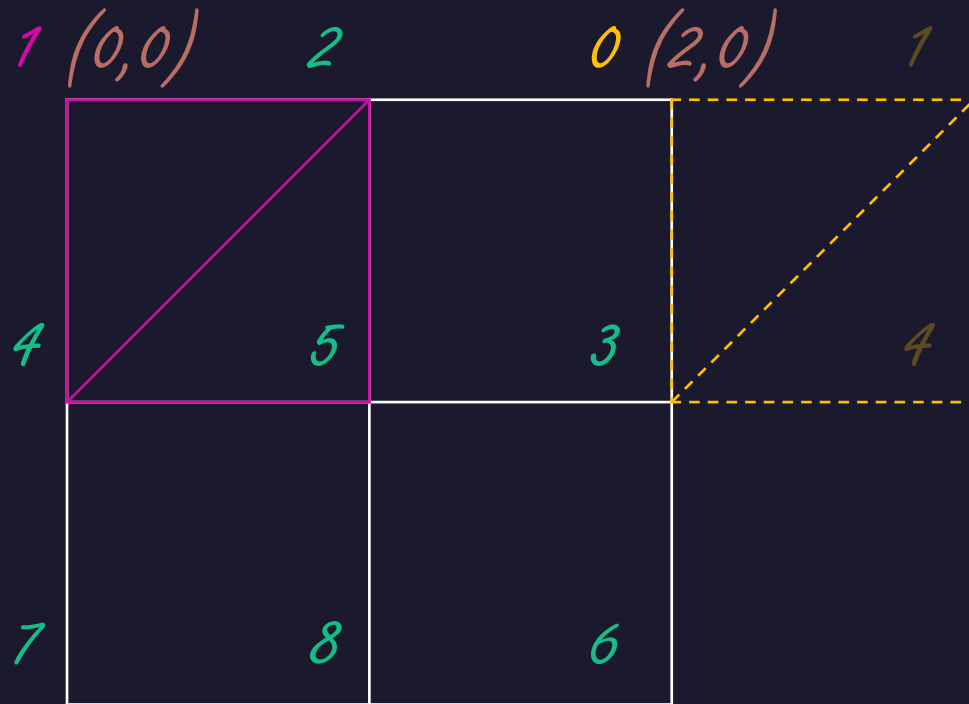
$$i = \text{round}((x - \text{camera}.x + \frac{W}{2})/S)$$
$$j = \text{round}((z - \text{camera}.z + \frac{W}{2})/S)$$

$$i' = i \% (N + 1)$$
$$j' = j \% (N + 1)$$

$$x' = x + \frac{i' - i}{N + 1} \cdot S \cdot (N + 1)$$

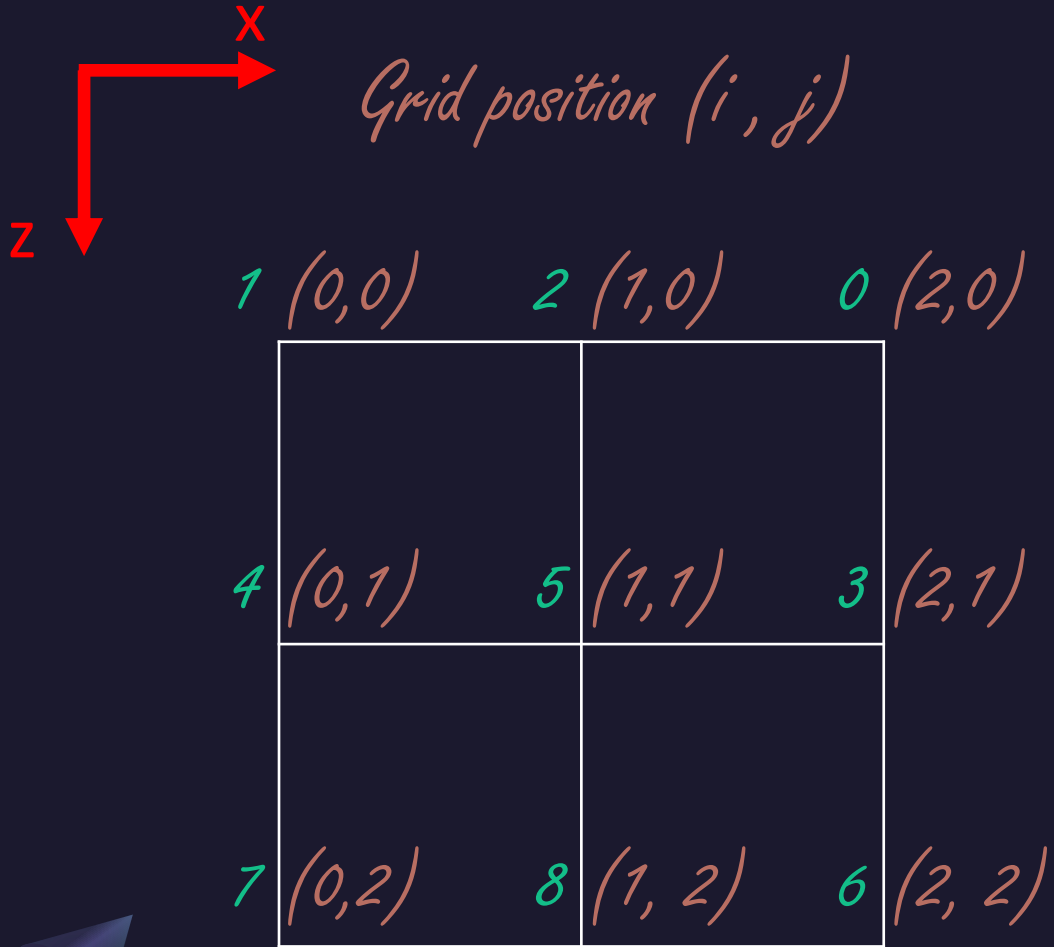
$$z' = z + \frac{j' - j}{N + 1} \cdot S \cdot (N + 1)$$

What about triangles ?

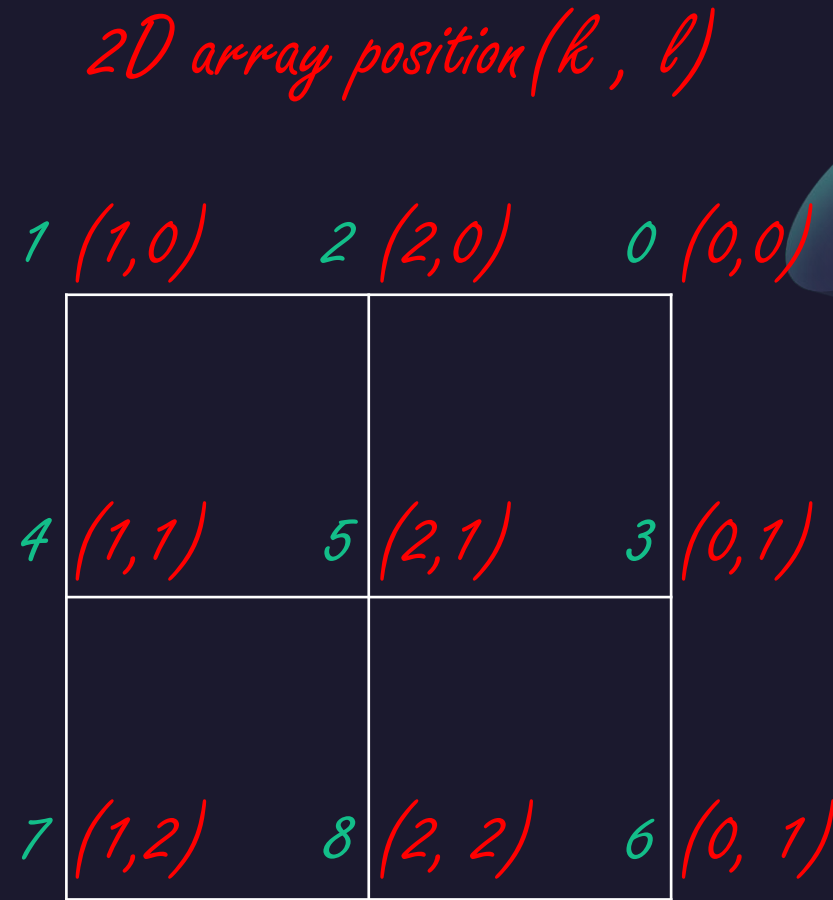


- $(1, 2, 4), (2, 4, 5)$ displayed
- $(0, 1, 3), (1, 3, 4)$ not displayed because vertex 0 on the right border

2 different positions

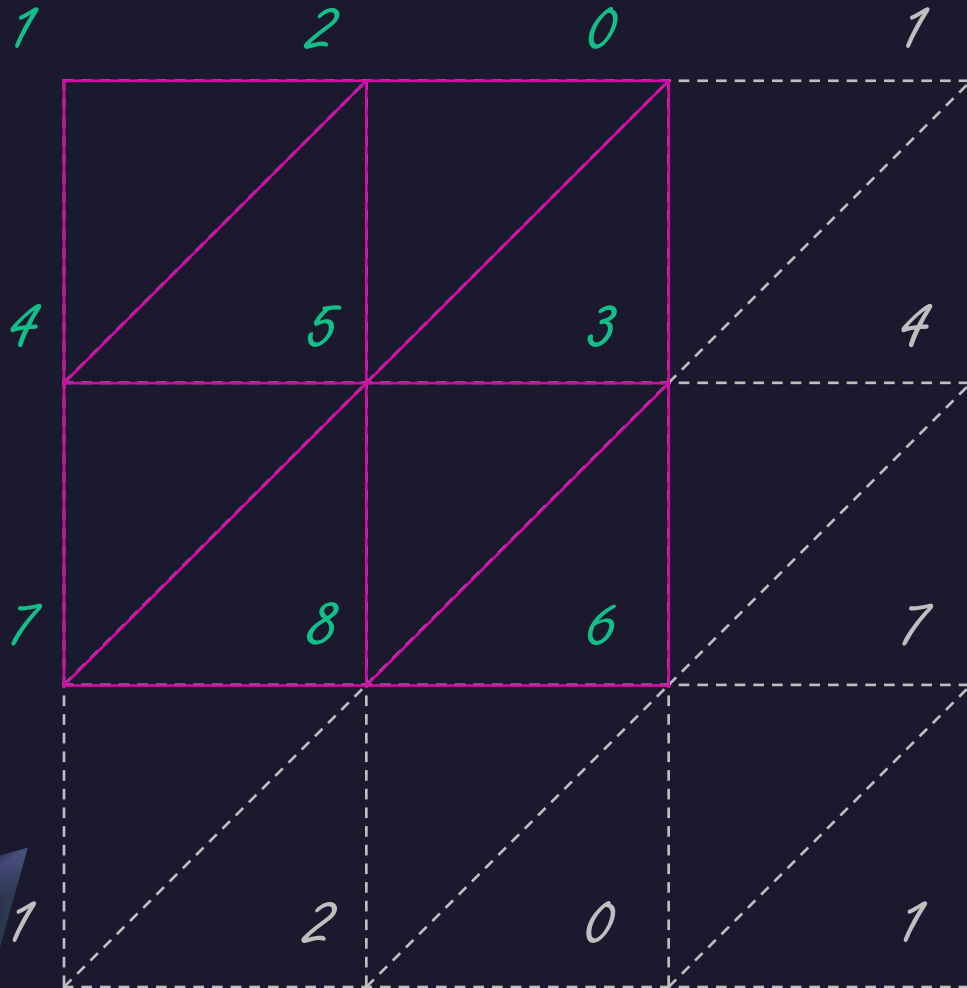


Virtual grid in 3D space
relative to vertex position



Actual structure containing vertices
relative to vertex index

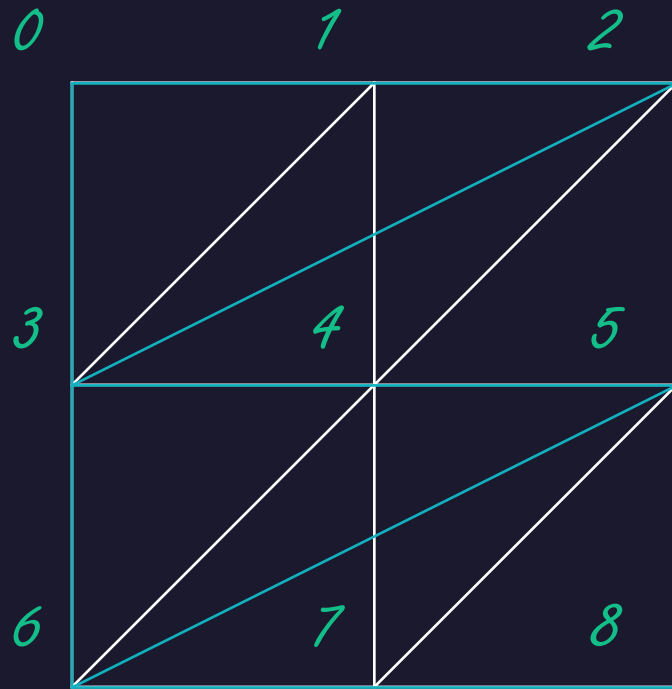
Displayed triangles



```
//(k,l) : 2D array indices
//(i,j) : grid position
//vertexIndex = 1 + k*(N+1)

if(i!= N and j!=N) {
    //compute neighbors indices
    v1 = 1 + k*(N+1)
    v2 = (1+1)%(N+1) + k*(N+1)
    v3 = 1 + ((k+1)%(N+1))*(N+1)
    v4 = (1+1)%(N+1) + ((k+1)%(N+1))*(N+1)
    show_triangle(v1, v2, v3)
    show_triangle(v2, v3, v4)
}
```

Triangles list



```
if (should_be_sidplayed[n]) {  
    triangles[2*n] = (v1, v2, v3)  
    triangles[2*n+1] = (v2, v3, v4)  
}  
else {  
    triangles[2*n] = (0, 0, 0)  
    triangles[2*n+1] = (0, 0, 0)  
}
```




PART V – Possible improvements

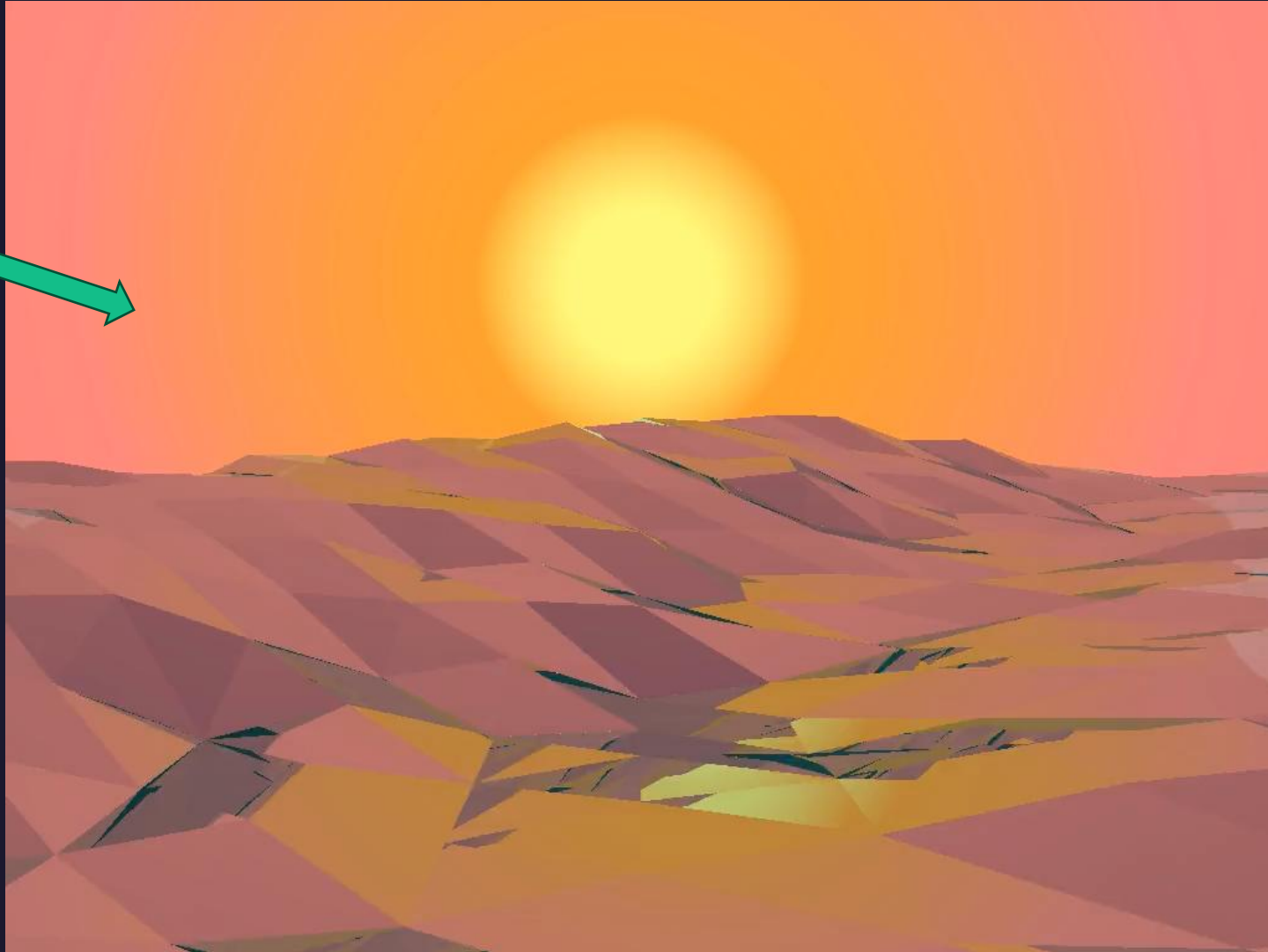
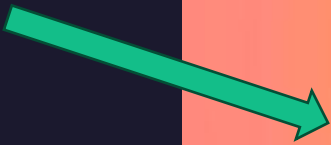


A larger sea



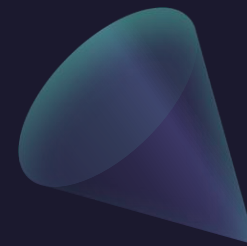
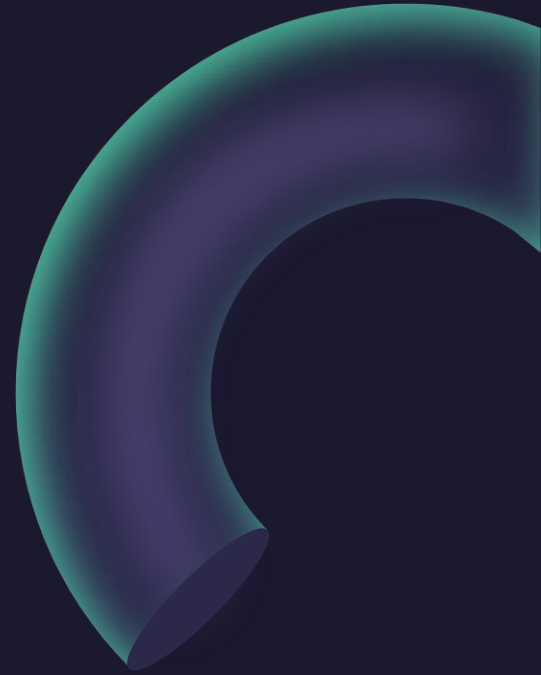
More triangles

Not in real time on
my laptop



Better performance

- Use Bounding box for the whole sea -> doesn't change much
- Use bounding boxes for parts of the sea -> more complicated, need to be recomputed every time the mesh changes
- Do all the computations related to simulation in a compute shader -> use the computing power of the GPU and save us the time to copy vertex data from CPU to GPU at each frame
- Copy only the displayed triangles into the storage buffer



Thank you

